EEET2601 Engineering Computing 1

Advanced Topics

Quang Tran

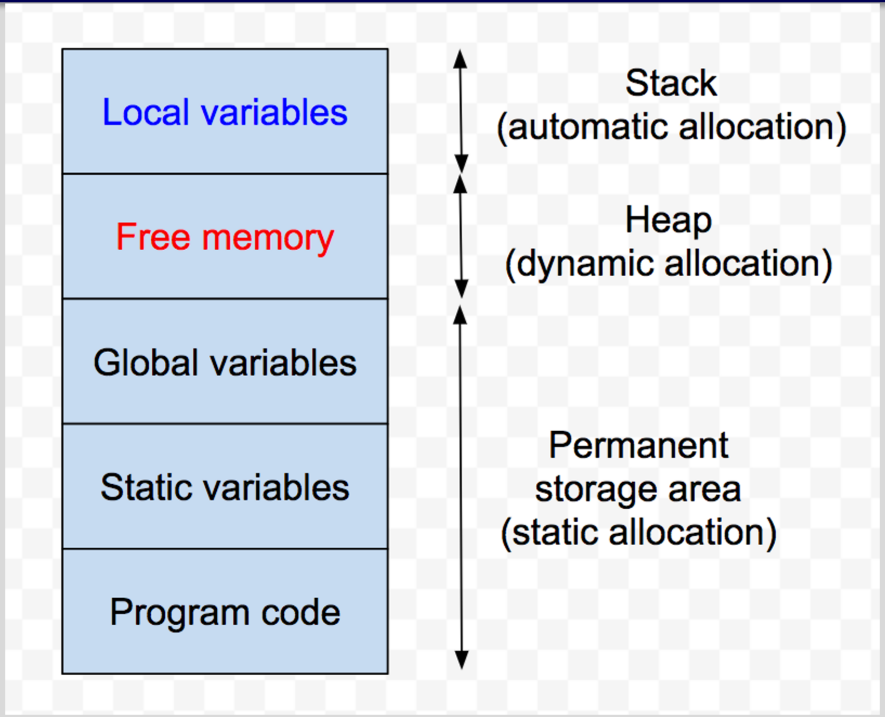
RMIT UNIVERSITY

Memory allocation

Three types of memory allocation in C:

1. Static memory allocation
2. Automatic memory allocation
3. Dynamic memory allocation

Memory allocation



Memory allocation

1. Static allocation: You can’t control this.
2. Automatic allocation: You can’t control this.
3. Dynamic allocation: You can control the exact size and lifetime of these memory allocations to achieve better efficiency. If you don’t free these memories after usage, you’ll run into memory leaks which may cause your program to crash or cannot allocate memory later.

Dynamic memory allocation functions in <stdlib.h>

Function	Description
void *malloc(int size);	Allocate **size** bytes, leave them uninitialized and return a void pointer pointing to the first byte of the allocated space. Return NULL if the allocation fails.
void *realloc(void *ptr, int new_size);	Reallocate the space pointed to by **ptr** up to **new_size**. Return NULL if the allocation fails.
void *calloc(int num, int size);	Allocate space for an array of **num** elements, each of **size** bytes. Initialize them to zero then return a void pointer to the memory. Return NULL if the allocation fails.
void free(void *ptr);	Release the memory pointed to by **ptr** back to the system

Example: Dynamic memory allocation

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char const *argv[]) {
    int num;
    printf("Enter the number of integers you want to store: ");
    scanf("%d", &num);

    int *ptr = (int *) malloc(num * sizeof(int));
    if (ptr == NULL) {
        printf("Sorry, cannot allocate that much memory");
        return 1;
    }

    // Read the integers into the dynamic memory area
    for (int i = 0; i < num; i++) {
        printf("Enter an integer: ");
        scanf("%i", ptr + i);
    }

    // Find the largest integer among them
    int largest = *ptr;
    for (int i = 1; i < num; i++) {
        if (*(ptr + i) > largest)
            largest = *(ptr + i);
    }

    // Free up the memory
    free(ptr);
    ptr = NULL; // Assign ptr to NULL to avoid potential errors

    printf("The largest integer is %i\n", largest);

    return 0;
}
```

Allocate a block of memory in heap that can be accessed via a pointer

Memory is accessed via a pointer

Free up the block of memory so that it can be used for other purposes

When to use dynamic memory allocation?

1. **When you need a lot of memory**
   - The size of the stack memory per process is usually between 1 to 8 MB, while the size of the heap memory is only limited by the available physical memory
2. **When you need the data after the function returns**
   - Stack memory gets destroyed when the function ends but heap memory is only freed when you want

When to use dynamic memory allocation?

3. **When you are building a data structure (e.g. an array of structure) of unknown size at compile time which could later become big**
   - Dynamic memory allocation allows you to request an exact amount of memory when you need at runtime

References

1. Fresh2Refresh, C Programming Tutorial, 2019.
2. StudyTonight, C Programming Language Tutorial, 2019.